Assignment 1

PROGRAM 1

Program Code

```
def main():
    try:
        n, k, m = map(int, input().split())
        students = []
        semesters = {}
        subjects = [[] for _ in range(m)]

        for _ in range(n):
            data = input().split()
            enrollment = data[0]
            name = data[1]
            semester = int(data[2])
            cpi = float(data[3])
            marks = list(map(int, data[4:]))

            student = (enrollment, name, semester, cpi, tuple(marks))
            students.append(student)

            if semester not in semesters:
                semesters[semester] = []
            semesters[semester].append(student)

            for i in range(m):
                subjects[i].append((marks[i], enrollment))

        for semester in sorted(semesters):
            group = semesters[semester]
            group.sort(key=lambda x: (-x[3], -sum(x[4]) / m, x[0]))
            result = [student[0] for student in group[:k]]
            print(f"Semester {semester}:", *result)

        for i in range(m):
            highest = max(mark for mark, enrollment in subjects[i])
            toppers = []
            for mark, enrollment in subjects[i]:
                if mark == highest:
                    toppers.append(enrollment)
            toppers.sort()
            print(f"S{i + 1}:", *toppers)

    except (ValueError, IndexError):
        print("INVALID INPUT")


if __name__ == "__main__":
    main()
```

Output

```
Semester 3: 2205 2203
Semester 5: 2202 2201
S1: 2203 2205
S2: 2202
S3: 2203
```

PROGRAM 2

Program Code

```
import re


def is_banned(password, banned_words):
    password = password.lower()
    for word in banned_words:
        if word.lower() in password:
            return True
    return False


def check_password(password, banned_words):
    if len(password) < 6 or len(password) > 12:
        return "WEAK_LENGTH"

    if is_banned(password, banned_words):
        return "COMPROMISED"

    if not re.search(r"[a-z]", password):
        return "WEAK_PATTERN"
    if not re.search(r"[A-Z]", password):
        return "WEAK_PATTERN"
    if not re.search(r"[0-9]", password):
        return "WEAK_PATTERN"
    if not re.search(r"[$#@]", password):
        return "WEAK_PATTERN"
    if re.search(r"(.)\1{3,}", password):
        return "WEAK_PATTERN"

    return "STRONG"


def main():
    try:
        b = int(input())
        banned_words = []

        for _ in range(b):
            banned_words.append(input().strip())

        n = int(input())

        for i in range(1, n + 1):
            password = input().rstrip("\n")
            result = check_password(password, banned_words)
            print(f"{i}: {result}")

    except (ValueError, EOFError):
        print("INVALID INPUT")


if __name__ == "__main__":
    main()
```

Output

```
1: STRONG
2: COMPROMISED
3: WEAK_PATTERN
4: WEAK_LENGTH
5: STRONG
```

PROGRAM 3

Program Code

```
import re
import sys

sys.setrecursionlimit(1000000)


class ExpressionError(Exception):
    pass


class Parser:
    def __init__(self, text, get_variable):
        self.text = text
        self.get_variable = get_variable
        self.tokens = re.findall(r"\d+|[A-Za-z_][A-Za-z0-9_]*|[+\-*()]", text)
        self.position = 0

        joined = "".join(self.tokens)
        original = re.sub(r"\s+", "", text)
        if joined != original:
            raise ExpressionError

    def current(self):
        if self.position < len(self.tokens):
            return self.tokens[self.position]
        return None

    def expression(self):
        value = self.term()
        while self.current() in ("+", "-"):
            operator = self.current()
            self.position += 1
            right = self.term()
            if operator == "+":
                value += right
            else:
                value -= right
        return value

    def term(self):
        value = self.factor()
        while self.current() == "*":
            self.position += 1
            value *= self.factor()
        return value

    def factor(self):
        token = self.current()

        if token is None:
            raise ExpressionError

        if token.isdigit():
            self.position += 1
            return int(token)

        if re.fullmatch(r"[A-Za-z_][A-Za-z0-9_]*", token):
            self.position += 1
            return self.get_variable(token)

        if token == "(":
            self.position += 1
            value = self.expression()
            if self.current() != ")":
                raise ExpressionError
            self.position += 1
            return value

        raise ExpressionError

    def parse(self):
        value = self.expression()
        if self.current() is not None:
            raise ExpressionError
        return value


def main():
    try:
        v = int(input())
        definitions = {}

        for _ in range(v):
            line = input()
            if "=" not in line:
                print("INVALID")
                return

            name, expression = line.split("=", 1)
```

```
            name = name.strip()
            expression = expression.strip()

            if not re.fullmatch(r"[A-Za-z_][A-Za-z0-9_]*", name):
                print("INVALID")
                return

            definitions[name] = expression

        target = input().strip()
        memo = {}
        visiting = set()

        def get_variable(name):
            if name in memo:
                return memo[name]
            if name not in definitions:
                raise ExpressionError
            if name in visiting:
                raise RuntimeError

            visiting.add(name)
            value = Parser(definitions[name], get_variable).parse()
            visiting.remove(name)
            memo[name] = value
            return value

        answer = Parser(target, get_variable).parse()
        print(answer)

    except RuntimeError:
        print("CYCLE")
    except (ValueError, ExpressionError, EOFError, RecursionError):
        print("INVALID")


if __name__ == "__main__":
    main()
```

Output

```
24
```

PROGRAM 4

Program Code

```
import csv
import os
import sys
from datetime import datetime


def validate_row(row):
    if len(row) < 5:
        raise ValueError("missing field")

    transaction_id = row[0].strip()
    account_id = row[1].strip()
    transaction_type = row[2].strip().upper()
    amount_text = row[3].strip()
    timestamp = row[4].strip()

    if transaction_id == "" or account_id == "" or timestamp == "":
        raise ValueError("missing field")

    if transaction_type not in ("CREDIT", "DEBIT"):
        raise ValueError("invalid transaction type")

    amount = float(amount_text)
    if amount <= 0:
        raise ValueError("amount must be greater than 0")

    datetime.fromisoformat(timestamp)

    return account_id, transaction_type, amount


def main():
    try:
        if len(sys.argv) > 1:
            filename = sys.argv[1]
        else:
            filename = input().strip()

        if not os.path.isfile(filename):
            print("INPUT FILE NOT FOUND")
            return

        credit_rows = []
        debit_rows = []
        error_rows = []
        balances = {}

        with open(filename, "r", newline="", encoding="utf-8") as file:
            reader = csv.reader(file)
            header = next(reader)

            for row in reader:
                try:
                    account, transaction_type, amount = validate_row(row)

                    if transaction_type == "CREDIT":
                        credit_rows.append(row)
                        balances[account] = balances.get(account, 0) + amount
                    else:
                        debit_rows.append(row)
                        balances[account] = balances.get(account, 0) - amount

                except (ValueError, IndexError) as error:
                    error_rows.append(row + [str(error)])

        with open("credit.csv", "w", newline="", encoding="utf-8") as file:
            writer = csv.writer(file)
            writer.writerow(header)
            writer.writerows(credit_rows)

        with open("debit.csv", "w", newline="", encoding="utf-8") as file:
            writer = csv.writer(file)
            writer.writerow(header)
            writer.writerows(debit_rows)

        with open("error.csv", "w", newline="", encoding="utf-8") as file:
            writer = csv.writer(file)
            writer.writerow(header + ["reason"])
            writer.writerows(error_rows)

        result = sorted(balances.items(), key=lambda x: (-abs(x[1]), x[0]))

        for account, balance in result:
            if balance.is_integer():
                balance = int(balance)
            print(account, balance)

        print("Files created: credit.csv, debit.csv, error.csv")
```

```
    except (OSError, StopIteration):
        print("INVALID INPUT")


if __name__ == "__main__":
    main()
```

Output

```
A1 500
A2 -100
Files created: credit.csv, debit.csv, error.csv
```

PROGRAM 5

Program Code

```
class BankError(Exception):
    pass


class Account:
    def __init__(self, account_id, balance):
        self.account_id = account_id
        self.balance = int(balance)

    def deposit(self, amount):
        if amount <= 0:
            raise BankError("Invalid amount")
        self.balance += amount

    def withdraw(self, amount):
        if amount <= 0:
            raise BankError("Invalid amount")
        if amount > self.balance:
            raise BankError("Insufficient funds")
        self.balance -= amount


class Transaction:
    def __init__(self, operation, account1, account2, amount):
        self.operation = operation
        self.account1 = account1
        self.account2 = account2
        self.amount = amount


class Bank:
    def __init__(self, accounts):
        self.accounts = accounts
        self.history = []

    def execute(self, transaction):
        if transaction.operation == "DEPOSIT":
            self.accounts[transaction.account1].deposit(transaction.amount)
        elif transaction.operation == "WITHDRAW":
            self.accounts[transaction.account1].withdraw(transaction.amount)
        elif transaction.operation == "TRANSFER":
            self.accounts[transaction.account1].withdraw(transaction.amount)
            self.accounts[transaction.account2].deposit(transaction.amount)
        else:
            raise BankError("Invalid operation")
        self.history.append(transaction)


def main():
    try:
        n = int(input())
        accounts = {}

        for _ in range(n):
            account_id, balance = input().split()
            accounts[account_id] = Account(account_id, int(balance))

        q = int(input())
        bank = Bank(accounts)
        batch_number = 0
        in_batch = False
        batch_failed = False
        old_balances = {}
        failed_batches = []

        for _ in range(q):
            parts = input().split()

            if parts[0] == "BATCH_BEGIN":
                batch_number += 1
                in_batch = True
                batch_failed = False
                old_balances = {}
                for key in accounts:
                    old_balances[key] = accounts[key].balance
                continue

            if parts[0] == "BATCH_END":
                if in_batch and batch_failed:
                    for key in accounts:
                        accounts[key].balance = old_balances[key]
                    failed_batches.append(batch_number)
                in_batch = False
                batch_failed = False
                continue

            if in_batch and batch_failed:
                continue
```

```
            try:
                if parts[0] == "DEPOSIT" and len(parts) == 3:
                    if parts[1] not in accounts:
                        raise BankError
                    transaction = Transaction("DEPOSIT", parts[1], "", int(parts[2]))
                    bank.execute(transaction)

                elif parts[0] == "WITHDRAW" and len(parts) == 3:
                    if parts[1] not in accounts:
                        raise BankError
                    transaction = Transaction("WITHDRAW", parts[1], "", int(parts[2]))
                    bank.execute(transaction)

                elif parts[0] == "TRANSFER" and len(parts) == 4:
                    if parts[1] not in accounts or parts[2] not in accounts:
                        raise BankError
                    transaction = Transaction("TRANSFER", parts[1], parts[2], int(parts[3]))
                    bank.execute(transaction)

                else:
                    raise BankError

            except (ValueError, BankError):
                if in_batch:
                    batch_failed = True

        if in_batch and batch_failed:
            for key in accounts:
                accounts[key].balance = old_balances[key]
            failed_batches.append(batch_number)

        for number in failed_batches:
            print("FAILED", number)

        for account_id in sorted(accounts):
            print(account_id, accounts[account_id].balance)

    except (ValueError, EOFError):
        print("INVALID INPUT")


if __name__ == "__main__":
    main()
```

Output

```
FAILED 1
A 700
B 500
```

PROGRAM 6

Program Code

```
import heapq


def find_cycle(graph):
    state = {}
    path = []
    position = {}

    for node in graph:
        state[node] = 0

    def dfs(node):
        state[node] = 1
        position[node] = len(path)
        path.append(node)

        for next_node in sorted(graph[node]):
            if state[next_node] == 0:
                result = dfs(next_node)
                if result:
                    return result
            elif state[next_node] == 1:
                return path[position[next_node]:] + [next_node]

        path.pop()
        position.pop(node)
        state[node] = 2
        return None

    for node in sorted(graph):
        if state[node] == 0:
            result = dfs(node)
            if result:
                return result

    return []


def main():
    try:
        n, e = map(int, input().split())
        modules = []

        for _ in range(n):
            modules.append(input().strip())

        graph = {}
        for module in modules:
            graph[module] = set()

        for _ in range(e):
            a, b = input().split()
            graph[a].add(b)

        dependency_count = {}
        reverse_graph = {}

        for module in modules:
            dependency_count[module] = len(graph[module])
            reverse_graph[module] = set()

        for module in graph:
            for dependency in graph[module]:
                reverse_graph[dependency].add(module)

        heap = []
        for module in modules:
            if dependency_count[module] == 0:
                heapq.heappush(heap, module)

        order = []

        while heap:
            module = heapq.heappop(heap)
            order.append(module)

            for next_module in reverse_graph[module]:
                dependency_count[next_module] -= 1
                if dependency_count[next_module] == 0:
                    heapq.heappush(heap, next_module)

        if len(order) == n:
            print(*order)
        else:
            cycle = find_cycle(graph)
            print("CYCLE")
            print(*cycle)

    except (ValueError, KeyError, EOFError):
```

```
        print("INVALID INPUT")

if __name__ == "__main__":
    main()
```

Output

```
core db auth ui
```

PROGRAM 7

Program Code

```
import re


class InvalidFormatError(Exception):
    pass


class UnknownVariableError(Exception):
    pass


class DivisionByZeroError(Exception):
    pass


class UnsupportedOperatorError(Exception):
    pass


def get_value(value, variables):
    if re.fullmatch(r"[+-]?(\d+(\.\d*)?|\.\d+)", value):
        return float(value)

    if re.fullmatch(r"[A-Za-z_][A-Za-z0-9_]*", value):
        if value not in variables:
            raise UnknownVariableError
        return variables[value]

    raise InvalidFormatError


def calculate(line, variables):
    parts = line.split()

    if len(parts) == 1:
        return get_value(parts[0], variables)

    if len(parts) != 3:
        raise InvalidFormatError

    left = get_value(parts[0], variables)
    operator = parts[1]
    right = get_value(parts[2], variables)

    if operator not in ["+", "-", "*", "/", "%"]:
        raise UnsupportedOperatorError

    if operator in ["/", "%"] and right == 0:
        raise DivisionByZeroError

    if operator == "+":
        return left + right
    if operator == "-":
        return left - right
    if operator == "*":
        return left * right
    if operator == "/":
        return left / right
    return left % right


def show(value):
    if value.is_integer():
        return int(value)
    return value


def main():
    variables = {}

    while True:
        try:
            line = input().strip()
        except EOFError:
            break

        if line.lower() == "quit":
            break

        try:
            assignment = re.fullmatch(r"([A-Za-z_][A-Za-z0-9_]*)\s*=\s*(.+)", line)

            if assignment:
                name = assignment.group(1)
                expression = assignment.group(2)
                variables[name] = calculate(expression, variables)
            else:
                result = calculate(line, variables)
```

```
                print(show(result))

        except InvalidFormatError:
            print("InvalidFormatError")
        except UnknownVariableError:
            print("UnknownVariableError")
        except DivisionByZeroError:
            print("DivisionByZeroError")
        except UnsupportedOperatorError:
            print("UnsupportedOperatorError")


if __name__ == "__main__":
    main()
```

Output

```
15
DivisionByZeroError
UnsupportedOperatorError
```

PROGRAM 8

Program Code

```
import os
import pickle
import re
import zipfile


def make_index(folder):
    index = {}
    files_count = 0
    lines_count = 0
    log_files = []

    for filename in sorted(os.listdir(folder)):
        if not filename.endswith((".txt", ".log")):
            continue

        path = os.path.join(folder, filename)
        if not os.path.isfile(path):
            continue

        log_files.append(path)
        files_count += 1

        with open(path, "r", encoding="utf-8", errors="ignore") as file:
            for line_number, line in enumerate(file, 1):
                lines_count += 1
                words = re.findall(r"[A-Za-z0-9_]+", line.lower())

                for word in set(words):
                    if word not in index:
                        index[word] = []
                    index[word].append((filename, line_number))

    return index, files_count, lines_count, log_files


def build(folder, archive_name):
    index, files_count, lines_count, log_files = make_index(folder)
    pickle_path = os.path.join(folder, "index.pkl")

    with open(pickle_path, "wb") as file:
        pickle.dump(index, file)

    with zipfile.ZipFile(archive_name, "w", zipfile.ZIP_DEFLATED) as archive:
        for path in log_files:
            archive.write(path, os.path.basename(path))
        archive.write(pickle_path, "index.pkl")

    print("FILES", files_count)
    print("LINES", lines_count)
    print("TOKENS", len(index))


def search(pickle_path, queries):
    with open(pickle_path, "rb") as file:
        index = pickle.load(file)

    for word in queries:
        print(word + ":")
        matches = index.get(word.lower(), [])
        for filename, line_number in matches:
            print(filename + ":" + str(line_number))


def main():
    try:
        data = input().split()
        mode = data[0].upper()

        if mode == "BUILD":
            folder = data[1]
            archive = data[2]
            build(folder, archive)

        elif mode == "SEARCH":
            pickle_path = data[1]
            q = int(data[2])
            queries = data[3:3 + q]
            search(pickle_path, queries)

        else:
            print("INVALID INPUT")

    except (IndexError, ValueError, OSError, pickle.PickleError):
        print("INVALID INPUT")


if __name__ == "__main__":
```

```
main()
```

Output

```
FILES 3
LINES 1200
TOKENS 845
```

PROGRAM 9

Program Code

```
import heapq


def schedule_jobs(workers, jobs):
    jobs.sort(key=lambda x: (x[0], x[5]))

    waiting = []
    worker_times = []

    for i in range(workers):
        heapq.heappush(worker_times, (0, i + 1))

    result = []
    time = 0
    index = 0
    total_wait = 0

    while index < len(jobs) or waiting:
        if not waiting and index < len(jobs):
            time = max(time, jobs[index][0])

        while index < len(jobs) and jobs[index][0] <= time:
            arrival, job_id, priority, duration, resources, order = jobs[index]
            heapq.heappush(waiting, (-priority, arrival, order, job_id, duration, resources))
            index += 1

        free_time, worker = heapq.heappop(worker_times)
        time = max(time, free_time)

        while index < len(jobs) and jobs[index][0] <= time:
            arrival, job_id, priority, duration, resources, order = jobs[index]
            heapq.heappush(waiting, (-priority, arrival, order, job_id, duration, resources))
            index += 1

        if not waiting:
            heapq.heappush(worker_times, (free_time, worker))
            continue

        priority, arrival, order, job_id, duration, resources = heapq.heappop(waiting)
        start = max(time, arrival, free_time)
        finish = start + duration
        wait = start - arrival
        total_wait += wait

        result.append((job_id, worker, start, finish))
        heapq.heappush(worker_times, (finish, worker))
        time = start

    result.sort(key=lambda x: x[0])
    return result, total_wait / len(jobs)


def main():
    try:
        workers, n = map(int, input().split())
        jobs = []

        for order in range(n):
            arrival, job_id, priority, duration, resources = input().split()
            jobs.append((int(arrival), job_id, int(priority), int(duration),
                int(resources), order))

        result, average_wait = schedule_jobs(workers, jobs)

        for job_id, worker, start, finish in result:
            print(job_id, "W" + str(worker), start, finish)

        print(f"AVG_WAIT {average_wait:.2f}")

    except (ValueError, EOFError):
        print("INVALID INPUT")


if __name__ == "__main__":
    main()
```

Output

```
J1 W1 0 5
J2 W2 1 4
J3 W2 4 6
AVG_WAIT 0.67
```

PROGRAM 10

Program Code

```
import csv
import json
import os
import tkinter as tk
from tkinter import filedialog, messagebox, ttk

DATA_FILE = "assignment_data.json"


class AssignmentTracker:
    def __init__(self, root):
        self.root = root
        self.root.title("Assignment Tracker")
        self.root.geometry("900x600")
        self.records = self.load_data()
        self.create_widgets()
        self.show_records()

    def load_data(self):
        if not os.path.exists(DATA_FILE):
            return []

        try:
            with open(DATA_FILE, "r") as file:
                return json.load(file)
        except:
            return []

    def save_data(self):
        with open(DATA_FILE, "w") as file:
            json.dump(self.records, file, indent=2)

    def create_widgets(self):
        frame = ttk.Frame(self.root, padding=10)
        frame.pack(fill="x")

        ttk.Label(frame, text="Enrollment").grid(row=0, column=0)
        self.enrollment = ttk.Entry(frame)
        self.enrollment.grid(row=0, column=1)

        ttk.Label(frame, text="Name").grid(row=0, column=2)
        self.name = ttk.Entry(frame)
        self.name.grid(row=0, column=3)

        ttk.Label(frame, text="Assignment").grid(row=1, column=0)
        self.assignment = ttk.Entry(frame)
        self.assignment.grid(row=1, column=1)

        ttk.Label(frame, text="Status").grid(row=1, column=2)
        self.status = ttk.Combobox(frame, values=["Pending", "Completed"], state="readonly")
        self.status.set("Pending")
        self.status.grid(row=1, column=3)

        ttk.Label(frame, text="Marks").grid(row=2, column=0)
        self.marks = ttk.Entry(frame)
        self.marks.grid(row=2, column=1)

        ttk.Label(frame, text="Remarks").grid(row=2, column=2)
        self.remarks = ttk.Entry(frame)
        self.remarks.grid(row=2, column=3)

        ttk.Button(frame, text="Add Student", command=self.add_record).grid(row=3, column=0)
        ttk.Button(frame, text="Filter", command=self.filter_records).grid(row=3, column=1)
        ttk.Button(frame, text="Show All", command=self.show_records).grid(row=3, column=2)
        ttk.Button(frame, text="Export CSV", command=self.export_csv).grid(row=3, column=3)

        columns = ("enrollment", "name", "assignment", "status", "marks", "remarks")
        self.table = ttk.Treeview(self.root, columns=columns, show="headings")
        for column in columns:
            self.table.heading(column, text=column.title())
            self.table.column(column, width=130)
        self.table.pack(fill="both", expand=True, padx=10, pady=10)

    def add_record(self):
        enrollment = self.enrollment.get().strip()
        name = self.name.get().strip()
        assignment = self.assignment.get().strip()
        marks = self.marks.get().strip()

        if enrollment == "" or name == "" or assignment == "":
            messagebox.showerror("Error", "Please fill all required fields")
            return

        if marks != "":
            try:
                marks = float(marks)
                if marks < 0 or marks > 100:
                    raise ValueError
```

```
            except ValueError:
                messagebox.showerror("Error", "Marks must be between 0 and 100")
                return

        record = {
            "enrollment": enrollment,
            "name": name,
            "assignment": assignment,
            "status": self.status.get(),
            "marks": marks,
            "remarks": self.remarks.get().strip()
        }

        self.records.append(record)
        self.save_data()
        self.show_records()
        messagebox.showinfo("Success", "Record saved")

    def show_records(self, records=None):
        if records is None:
            records = self.records

        for item in self.table.get_children():
            self.table.delete(item)

        for record in records:
            values = (
                record["enrollment"],
                record["name"],
                record["assignment"],
                record["status"],
                record["marks"],
                record["remarks"]
            )
            self.table.insert("", "end", values=values)

    def filter_records(self):
        status = self.status.get()
        records = []

        for record in self.records:
            if record["status"] == status:
                records.append(record)

        self.show_records(records)

    def export_csv(self):
        filename = filedialog.asksaveasfilename(
            defaultextension=".csv",
            filetypes=[("CSV files", "*.csv")]
        )

        if filename == "":
            return

        fields = ["enrollment", "name", "assignment", "status", "marks", "remarks"]

        with open(filename, "w", newline="", encoding="utf-8") as file:
            writer = csv.DictWriter(file, fieldnames=fields)
            writer.writeheader()
            writer.writerows(self.records)

        messagebox.showinfo("Success", "CSV file exported")


def main():
    root = tk.Tk()
    AssignmentTracker(root)
    root.mainloop()


if __name__ == "__main__":
    main()
```

Output



The program opens a graphical Assignment Tracker window. The screenshot above shows the interface and sample records.

Assignment 2